INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Proyecto Playlist
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3-B
Alumnos participantes:	Chiluisa Estiven Ortiz Jeampierre Rodriguez David Toapanta Pablo
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caizabuano Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con listas secuenciales

Específicos:

1.1 Diseñar las clases de los Nodos y las Listas utilizando plantillas (templates) en C++ para permitir que las estructuras almacenen distintos tipos de datos (como std::string para nombres e int para IDs).

1.2 Implementar los algoritmos de inserción, búsqueda, eliminación y recorridos (unidireccionales y bidireccionales), garantizando el control exacto y sin fugas de memoria de los punteros primero, ultimo, siguiente y anterior.

1.3 Comprobar la correcta ejecución de las operaciones mediante un menú interactivo en consola que simule el caso de estudio de las playlists, demostrando que el programa no colapsa al reconectar los enlaces de los nodos durante las eliminaciones o inserciones.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Revise la documentación de cómo utilizar la interfaz
2. Revise la documentación de cómo utilizar la clase Stack en:
3.Diseñe la solución al caso de estudio adjunto.
4. Implemente la solución propuesta
5. Elabore un informe del trabajo realizado. Incluya en él el diagrama de clases que incluya las clases fundamentales de su propuesta de solución.
6. Suba el informe a la plataforma.

Nota: solo un integrante de cada grupo deberá realizar la entrega.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- PC de escritorio o portátil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación

☐Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

Comparar la implementación de pilas y colas utilizando las clases e interfaces que provee Java

2.7 Resultados obtenidos

Link del repositorio:
https://github.com/PabloToapanta/APE1_GRUPO_04_ESTRUCTURA_DATOS.git

El informe se encuentra en ANEXO 1
La consulta se encuentra en el ANEXO 2

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☒ Trabajo en equipo
☒ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☒ La resolución de conflictos
☒ Adaptabilidad
☐ Responsabilidad

2.9 Conclusiones

El desarrollo de este trabajo permitió comprender mejor la importancia del manejo correcto de la memoria y de los punteros al trabajar con estructuras dinámicas. Se observó que las operaciones de inserción, ya sea al inicio o al final de la estructura, así como las operaciones de eliminación, representan uno de los mayores retos técnicos. Esto se debe a que requieren un control cuidadoso de los punteros como primero, ultimo, siguiente y anterior. Si alguno de estos enlaces no se actualiza correctamente o si la memoria de un nodo eliminado no se libera adecuadamente con delete, la estructura puede corromperse, perder información o incluso provocar errores graves en la ejecución del programa. En este contexto, el uso del puntero ultimo resultó ser una estrategia muy útil para hacer más eficientes las inserciones al final de la lista.

Durante la implementación también se logró entender de forma más clara cómo funcionan las clases genéricas en C++ mediante plantillas (templates). A lo largo del proceso se comprobó que, para convertir una clase tradicional en una clase genérica, es necesario declarar template <typename T> antes de la definición de la clase y de sus métodos. Esto permite que el compilador interprete la clase como una especie de “molde” que puede adaptarse a distintos tipos de datos. De esta manera, el tipo concreto —como int o std::string— se define únicamente cuando la clase se utiliza o se instancia en el programa principal.

2.10 Recomendaciones

Ninguna

2.11 Referencias bibliográficas

[1] “Templates in C++ - GeeksforGeeks.” Accessed: Mar. 22, 2026. [Online]. Available: https://www.geeksforgeeks.org/cpp/templates-cpp/

[2] “C++ Program to Create an Interface - GeeksforGeeks.” Accessed: Mar. 22, 2026. [Online]. Available: https://www.geeksforgeeks.org/cpp/cpp-program-to-create-an-interface/

[3] "Stack in C++ STL - GeeksforGeeks." Accessed: Mar. 22, 2026. [Online]. Available: https://www.geeksforgeeks.org/cpp/stack-in-cpp-stl/

[4] "Forward List in C++ STL - GeeksforGeeks." Accessed: Mar. 22, 2026. [Online]. Available: https://www.geeksforgeeks.org/cpp/forward-list-c-set-1-introduction-important-functions/

2.12 Anexos

ANEXO 1: INFORME DEL DEBER

Objetivo del proyecto

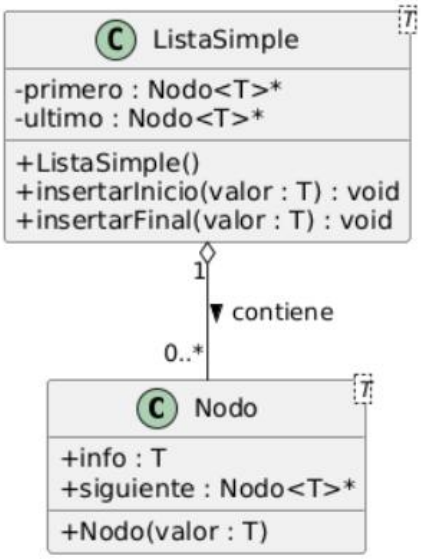
El objetivo de este proyecto es implementar dos estructuras de datos dinámicas lineales en C++: una Lista Simplemente Enlazada y una Lista Doblemente Enlazada. Ambas estructuras fueron desarrolladas utilizando plantillas (templates) para garantizar su genericidad, permitiendo almacenar cualquier tipo de dato (por ejemplo, std::string para nombres de canciones y tipos enteros int para los IDs). El caso práctico simulado es un Gestor de Playlists Musicales.

EQUIPO 1

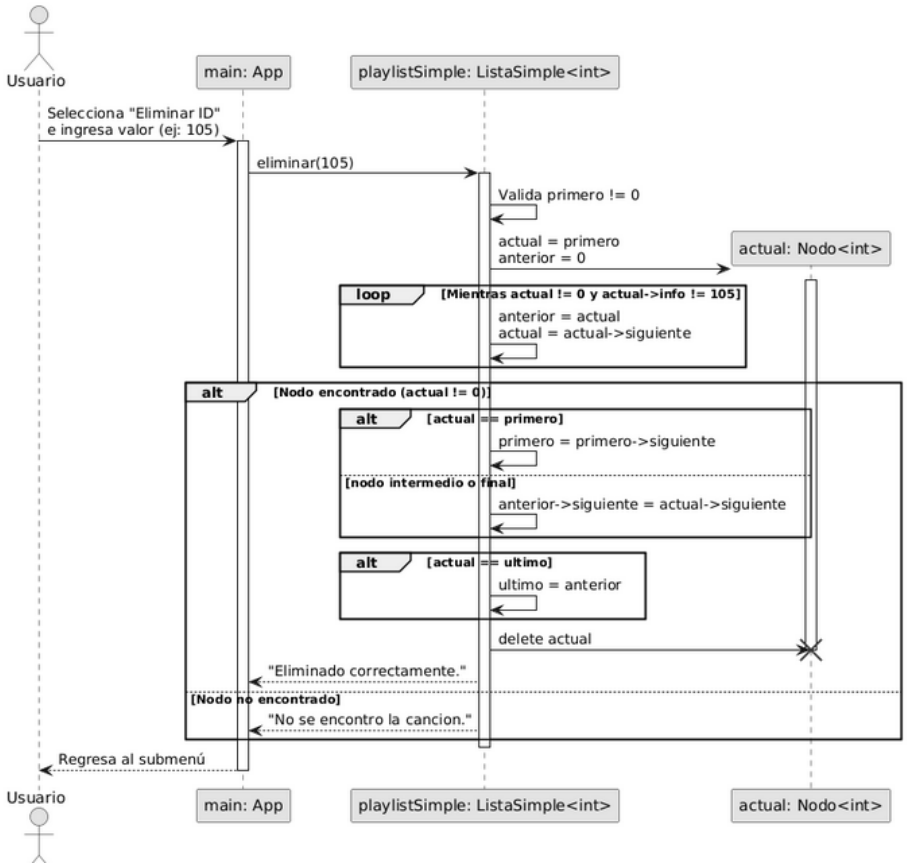
1. Diagrama de clases UML de las clases nodo y listas simple

Este diagrama representa de forma clara la organización y la relación que existe entre las clases Nodo<T> y ListaSimple<T>, tomando en cuenta sus atributos y métodos principales. Muestra cómo la lista simple se encarga de gestionar un conjunto de nodos donde cada uno almacena información y una referencia al siguiente elemento.

Esto facilita la comprensión del diseño del programa antes de su ejecución, ya que permite ver cómo interactúan sus componentes, mejorar la organización y entender el código de una manera más visual



2. **Diagrama de Secuencia (Operación Eliminar ID)**



3. **Explicación de las operaciones**
   - Método **InsertarInicio ()**

```
void insertarInicio(T valor) {
    // TODO Equipo 1: Lógica aquí
    Nodo<T>* nuevo = new Nodo<T>(valor); // Creamos el nuevo nodo en memoria

    // Si la lista está vacía, el nuevo nodo es tanto el primero como el último
    if (primero == 0) {
        primero = nuevo;
        ultimo = nuevo;
    } else {
        // Si ya hay nodos, el 'siguiente' del nuevo nodo apunta al que antes era el
        nuevo->siguiente = primero;
        primero = nuevo; // Ahora el primero es oficialmente el nuevo nodo
    }
}
```

Este método funciona agregando un nuevo nodo al comienzo de la lista por lo que el nuevo nodo se enlaza al nodo que actualmente es el primero, para que posteriormente el primer puntero se actualice y apunte a un nuevo nodo

- Método **insertarFinal ()**

```
void insertarFinal(T valor) {
    Nodo<T>* nuevo = new Nodo<T>(valor);

    // Si la lista está vacía, la lógica es idéntica a insertar al inicio
    if (primero == 0) {
        primero = nuevo;
        ultimo = nuevo;
    } else {
        // Aquí se cumple la regla de actualizar el puntero 'ultimo'
        ultimo->siguiente = nuevo; // El último actual ahora se enlaza al nuevo nodo
        ultimo = nuevo;            // El nuevo nodo se convierte en el último oficial
    }
}
```

Este método hace uso del puntero ultimo debido a que, en lugar de recorrer toda la lista, re realiza una inversión directa en tiempo constante pasando de una complejidad de O(n) a O(1). Esto se logra conectando el nuevo nodo al campo siguiente del nodo final actual y luego actualizando el puntero ultimo para que apunte al nuevo nodo.
En ambos métodos, se puede observar el caso en el que la lista está vacía. En esta situación, se garantiza la correcta inicialización de la estructura haciendo que tanto el puntero primero como ultimo apunte al mismo nodo que será el único elemento de la lista en ese momento.

Método buscar

```
bool buscar(T valor) {
    Nodo<T>* actual = primero;
    while (actual != nullptr) {
        if (actual->info == valor) return true;
        actual = actual->siguiente;
    }
    return false;
}
```

Realiza un recorrido secuencial (búsqueda lineal) comenzando desde el puntero primero. Evalúa el atributo info de cada nodo iterando a través del puntero siguiente; si encuentra una coincidencia con el ID numérico proporcionado, detiene la ejecución y retorna true. Si alcanza el final de la lista (0 o NULL) sin éxito, retorna false.

Método eliminar

```
// 2. Eliminar valor (Cuidado con los punteros)
void eliminar(T valor) {
    if (primero == nullptr) return;
    Nodo<T>* actual = primero;
    Nodo<T>* anterior = nullptr;

    while (actual != nullptr && actual->info != valor) {
        anterior = actual;
        actual = actual->siguiente;
    }

    if (actual != nullptr) {
        if (actual == primero) {
            primero = primero->siguiente;
            if (primero == nullptr) ultimo = nullptr;
        } else {
            anterior->siguiente = actual->siguiente;
            if (actual == ultimo) ultimo = anterior;
        }
        delete actual;
        cout << "Eliminado correctamente." << endl;
    }
}
```

Es una operación crítica que requiere reasignación de memoria. Utiliza dos punteros auxiliares (actual y anterior) para rastrear la posición. Al encontrar el nodo objetivo, el algoritmo "puentea" el enlace (anterior->siguiente = actual->siguiente). Contiene validaciones estrictas para tres casos límite:

1. Si se elimina el primer elemento, se desplaza el puntero primero.
2. Si se elimina el único elemento existente, ambos punteros (primero y ultimo) se vuelven nulos.
3. Si se elimina el último nodo, el puntero ultimo retrocede y se ancla en el nodo anterior. Finalmente, se libera el espacio en memoria usando la directiva delete.

Método mostrarLista

```
void mostrarLista() {
    Nodo<T>* actual = primero;
    while (actual != nullptr) {
        cout << actual->info << " -> ";
        actual = actual->siguiente;
    }
    cout << "NULL" << endl;
}
```

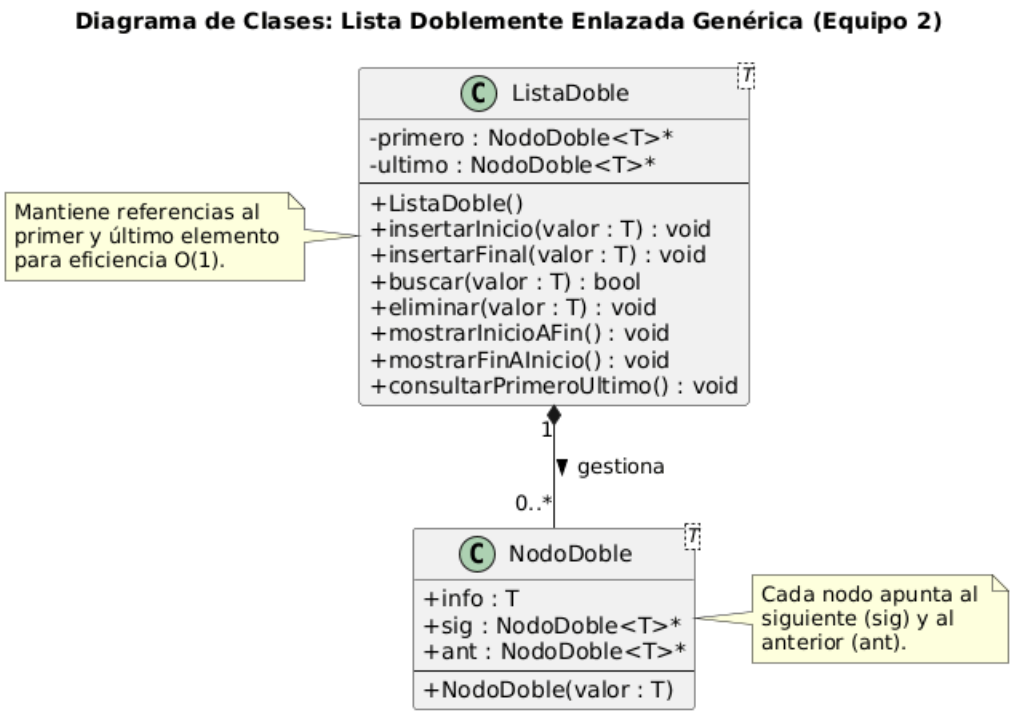
Recorre la estructura unidireccionalmente desde el nodo primero. Mientras avanza usando el enlace siguiente, imprime en consola el valor almacenado, separando cada elemento con una flecha (->) para ilustrar gráficamente la naturaleza de una lista simplemente enlazada hasta llegar al puntero NULL.

Método consultarPrimeroUltimo

```
// 4. Consultar primero y ultimo
void consultarPrimeroUltimo() {
    if (primero != nullptr && ultimo != nullptr) {
        cout << "Primero: " << primero->info << ", Ultimo: " << ultimo->info << endl;
    }
}
```

Aprovecha el diseño estructural de la clase que mantiene el registro de los extremos de la lista. En lugar de iterar sobre los elementos (lo que tomaría tiempo lineal), esta función accede directamente a los atributos info de los punteros primero y ultimo en tiempo de ejecución constante, verificando previamente que la lista no esté vacía para prevenir excepciones de memoria.

EQUIPO 2
DIAGRAMA DE CLASES
Lista Simple



EXPLICACICON DE OPERACIONES FUNDAMENTALES

Operaciones en la Lista Simple (Equipo 1):

- **Inserción al Final:** Para mantener la eficiencia de O(1), la clase cuenta con un puntero ultimo. Al insertar, en lugar de recorrer toda la lista, simplemente se enlaza el nuevo nodo al siguiente del puntero ultimo y luego se actualiza ultimo para que apunte al nuevo nodo ingresado.
- **Eliminación:** Se realiza un recorrido secuencial manteniendo un puntero auxiliar anterior. Al encontrar el nodo a eliminar, se "puentea" el enlace (anterior->siguiente = actual->siguiente). Si el nodo eliminado era el primero o el último, se actualizan los punteros principales respectivamente y se libera la memoria con delete.

Operaciones en la Lista Doble (Equipo 2):

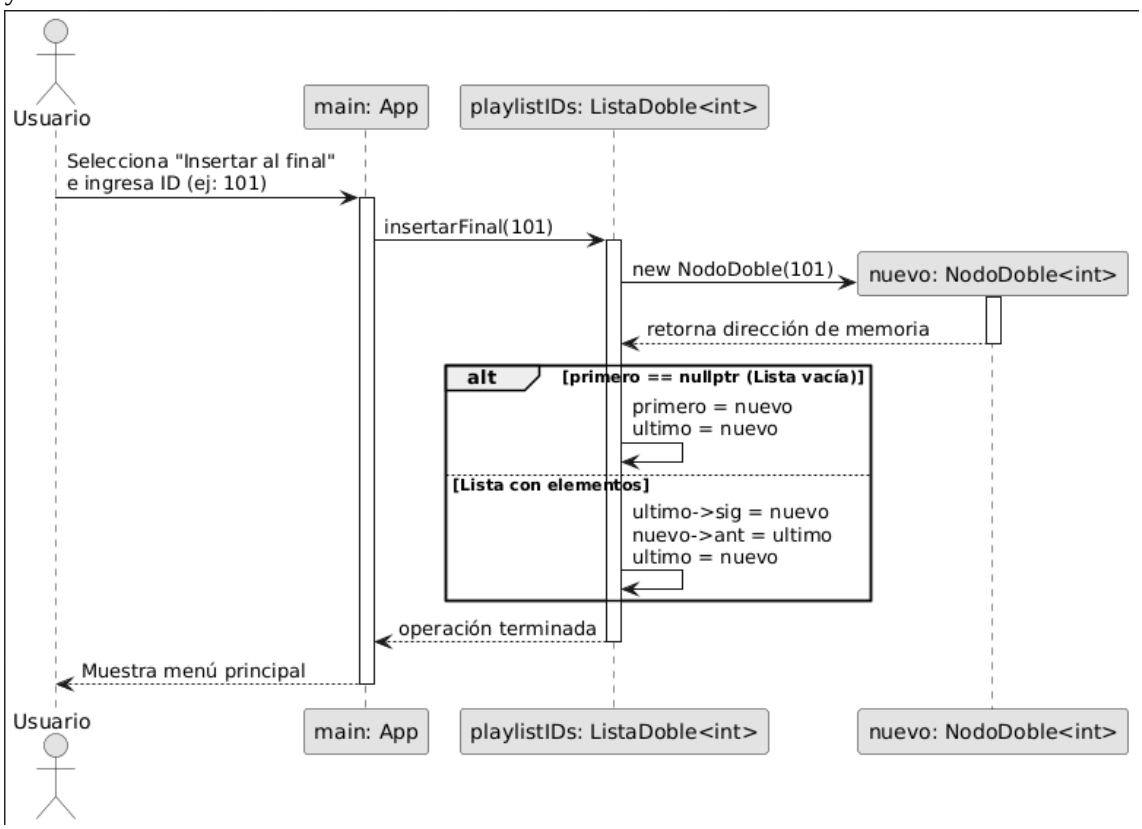
- **Inserciones (Doble Enlace):** A diferencia de la lista simple, aquí cada nuevo nodo (nuevo) debe reconocer a su antecesor (ant) y a su sucesor (sig). Al insertar al inicio o al

final, la lógica principal radica en cruzar los punteros correctamente (ej: nuevo->ant = ultimo; ultimo->sig = nuevo; ultimo = nuevo;).

- **Recorridos (Inicio a Fin / Fin a Inicio):** Gracias al doble enlace, la lista permite una navegación bidireccional. El método mostrarInicioAFin() inicia en el puntero primero y avanza usando el puntero sig. El método mostrarFinAInicio() arranca en el puntero ultimo y retrocede utilizando el puntero ant, validando en cada iteración que el nodo actual no sea nulo.

DIAGRAMA DE SECUENCIA

El diagrama de secuencia ilustra el flujo de la operación insertarFinal en la lista doblemente enlazada. Destaca la creación dinámica del objeto NodoDoble y la bifurcación lógica (alt) donde el sistema decide si actualizar ambos punteros iniciales (cuando la lista está vacía) o reconectar los punteros sig y ant del último elemento existente, demostrando un uso eficiente de la memoria y las referencias.



RESULTADOS OBTENIDOS

EQUIPO 1

El sistema compila y se ejecuta correctamente en consola sin errores de memoria. Se utilizaron tipos genéricos (int) para validar el funcionamiento de las plantillas.

 Captura **1:** Mostrando el menú principal y la inserción de un par de canciones.

```
--- GESTOR DE PLAYLIST SIMPLE (Equipo 1) ---
1. Insertar ID de cancion al INICIO
2. Insertar ID de cancion al FINAL
3. Buscar ID de cancion
4. Eliminar ID de cancion
5. Mostrar Playlist
6. Ver Primero y Ultimo
0. Volver al Menu Principal
Elija una opcion: 1
Ingrese el ID de la cancion (numero): 23

--- GESTOR DE PLAYLIST SIMPLE (Equipo 1) ---
1. Insertar ID de cancion al INICIO
2. Insertar ID de cancion al FINAL
3. Buscar ID de cancion
4. Eliminar ID de cancion
5. Mostrar Playlist
6. Ver Primero y Ultimo
0. Volver al Menu Principal
Elija una opcion: 1
Ingrese el ID de la cancion (numero): 34

--- GESTOR DE PLAYLIST SIMPLE (Equipo 1) ---
1. Insertar ID de cancion al INICIO
2. Insertar ID de cancion al FINAL
3. Buscar ID de cancion
4. Eliminar ID de cancion
5. Mostrar Playlist
6. Ver Primero y Ultimo
0. Volver al Menu Principal
Elija una opcion: 5
34 -> 23 -> NULL
```

Captura **2:** Mostrando la opción Biscar ID de la canción

```
--- GESTOR DE PLAYLIST SIMPLE (Equipo 1) ---
1. Insertar ID de cancion al INICIO
2. Insertar ID de cancion al FINAL
3. Buscar ID de cancion
4. Eliminar ID de cancion
5. Mostrar Playlist
6. Ver Primero y Ultimo
0. Volver al Menu Principal
Elija una opcion: 3
ID a buscar: 34
┬íSi esta en la lista!
```

Captura 3: Mostrar toda la lista de IDs

```
--- GESTOR DE PLAYLIST SIMPLE (Equipo 1) ---
1. Insertar ID de cancion al INICIO
2. Insertar ID de cancion al FINAL
3. Buscar ID de cancion
4. Eliminar ID de cancion
5. Mostrar Playlist
6. Ver Primero y Ultimo
0. Volver al Menu Principal
Elija una opcion: 5
34 -> 23 -> NULL
```

EQUIPO 2

El sistema compila y se ejecuta correctamente en consola sin errores de memoria. Se utilizaron tipos genéricos (std::string) para validar el funcionamiento de las plantillas.

 Captura **1:** Mostrando el menú principal y la inserción de un par de canciones.

```
--- GESTOR DE PLAYLIST (Equipo 2) ---
1. Insertar cancion al INICIO
2. Insertar cancion al FINAL
3. Buscar cancion
4. Eliminar cancion
5. Mostrar de Inicio a Fin
6. Mostrar de Fin a Inicio
7. Ver Primero y Ultimo
0. Salir
Elija una opcion: 1
Nombre de la cancion: Si tu me quisieras

--- GESTOR DE PLAYLIST (Equipo 2) ---
1. Insertar cancion al INICIO
2. Insertar cancion al FINAL
3. Buscar cancion
4. Eliminar cancion
5. Mostrar de Inicio a Fin
6. Mostrar de Fin a Inicio
7. Ver Primero y Ultimo
0. Salir
Elija una opcion: 2
Nombre de la cancion: Mami 100pre sabe

--- GESTOR DE PLAYLIST (Equipo 2) ---
1. Insertar cancion al INICIO
2. Insertar cancion al FINAL
3. Buscar cancion
4. Eliminar cancion
5. Mostrar de Inicio a Fin
6. Mostrar de Fin a Inicio
7. Ver Primero y Ultimo
0. Salir
Elija una opcion: |
```

 Captura **2:** Mostrando la opción "Mostrar de Inicio a Fin" y "Mostrar de Fin a Inicio" para demostrar que los enlaces dobles funcionan.

```
--- GESTOR DE PLAYLIST (Equipo 2) ---
1. Insertar cancion al INICIO
2. Insertar cancion al FINAL
3. Buscar cancion
4. Eliminar cancion
5. Mostrar de Inicio a Fin
6. Mostrar de Fin a Inicio
7. Ver Primero y Ultimo
0. Salir
Elija una opcion: 5
Inicio -> Fin: Si tu me quisieras <-> Mami 100pre sabe

--- GESTOR DE PLAYLIST (Equipo 2) ---
1. Insertar cancion al INICIO
2. Insertar cancion al FINAL
3. Buscar cancion
4. Eliminar cancion
5. Mostrar de Inicio a Fin
6. Mostrar de Fin a Inicio
7. Ver Primero y Ultimo
0. Salir
Elija una opcion: 6
Fin -> Inicio: Mami 100pre sabe <-> Si tu me quisieras
```

 Captura **3:** Ejecutando la opción "Eliminar canción" y mostrando nuevamente la lista para evidenciar que el elemento desapareció y el programa no colapsó.

```
--- GESTOR DE PLAYLIST (Equipo 2) ---
1. Insertar cancion al INICIO
2. Insertar cancion al FINAL
3. Buscar cancion
4. Eliminar cancion
5. Mostrar de Inicio a Fin
6. Mostrar de Fin a Inicio
7. Ver Primero y Ultimo
0. Salir
Elija una opcion: 4
Cancion a eliminar: Si tu me quisieras

--- GESTOR DE PLAYLIST (Equipo 2) ---
1. Insertar cancion al INICIO
2. Insertar cancion al FINAL
3. Buscar cancion
4. Eliminar cancion
5. Mostrar de Inicio a Fin
6. Mostrar de Fin a Inicio
7. Ver Primero y Ultimo
0. Salir
Elija una opcion: 5
Inicio -> Fin: Mami 100pre sabe
```

ANEXO 2: REVISION DE LA DOCUMENTACION

TEMPLATES

Una plantilla C++ es una herramienta para crear clases o funciones genéricas. Esto nos permite escribir código que funciona para cualquier tipo de datos sin reescribirlo para cada tipo, se caracteriza porque evita la duplicación de código permitiendo que una función o clase funcione con múltiples tipos de datos, principalmente permitiendo funciones y clases genéricas, además de proporcionar seguridad de tipo a diferencia de los punteros o macros void*, y puede especializarse para tipos de datos específicos cuando sea necesario.[1]

Ejemplo

Example:

```
#include <iostream>
using namespace std;

template <typename T> T myMax(T x, T y){
    return (x > y) ? x : y;
}

int main(){

    cout << "Max of 3 and 7 is: " << myMax<int>(3, 7) << endl;
    cout << "Max of 3.5 and 7.5 is :" << myMax<double>(3.5, 7.5) << endl;
    cout << "Max of 'g' and 'e' is: " << myMax<char>('g', 'e') << endl;

    return 0;
}
```

Output

```
Max of 3 and 7 is: 7
Max of 3.5 and 7.5 is :7.5
Max of 'g' and 'e' is: g
```

INTERFACES EN C++

Las interfaces en Java son una característica que permite definir un tipo abstracto que describe el comportamiento que debe tener una clase. Es decir, establecen qué métodos debe tener una clase, pero no cómo deben implementarse.[2]

En C++, no existe el concepto de interfaz como tal. Sin embargo, es posible lograr algo muy similar utilizando clases abstractas puras (pure abstract classes).

Para entender cómo se logra esto, primero hay que considerar cómo funcionan las interfaces. Una interfaz únicamente declara los métodos que deben existir, pero no incluye su implementación. Además, esos métodos se definen como públicos, cualquier clase puede implementar una interfaz y la clase que la implemente está obligada a proporcionar el código de los métodos declarados. Otra característica importante es que no se pueden crear objetos directamente a partir de una interfaz.[2]

En C++, estas mismas ideas se pueden representar mediante una clase abstracta pura, que es una clase formada únicamente por funciones virtuales puras. En este tipo de clases los métodos se declaran, pero no se definen dentro de la clase. Sus métodos pueden tener diferentes niveles de acceso, como público, protegido o privado, y cualquier otra clase puede heredar de ella.

Cuando una clase hereda de una clase abstracta pura, debe implementar todos los métodos que fueron declarados. Si no lo hace, esa clase también se considerará abstracta y tampoco podrá instanciarse. De manera similar a las interfaces, tampoco es posible crear objetos directamente a partir de una clase abstracta pura.[2]

Ejemplo

```
// Interface equivalent pure abstract class
class I {
  public:
    virtual string getName() = 0;
};

// Class B which inherits I
class B : public I {
  public:
    string getName() {
        return "GFG";
    }
};

// Class C which inherits I
class C : public I {
  public:
    string getName() {
        return "GeeksforGeeks";
    }
};
```

```
int main() {
    B obj1;
    C obj2;
    I *ptr;

    // Assigning the address of obj1 to ptr
    ptr = &obj1;
    cout << ptr->getName() << endl;

    // Assigning the address of obj2 to ptr
    ptr = &obj2;
    cout << ptr->getName();

    return 0;
}
```

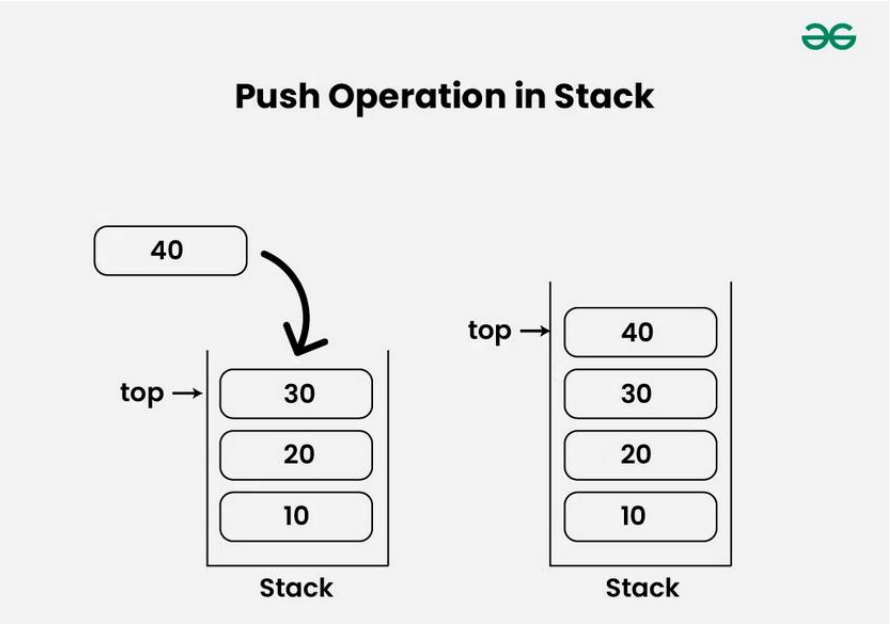
Output

```
GFG
GeeksforGeeks
```

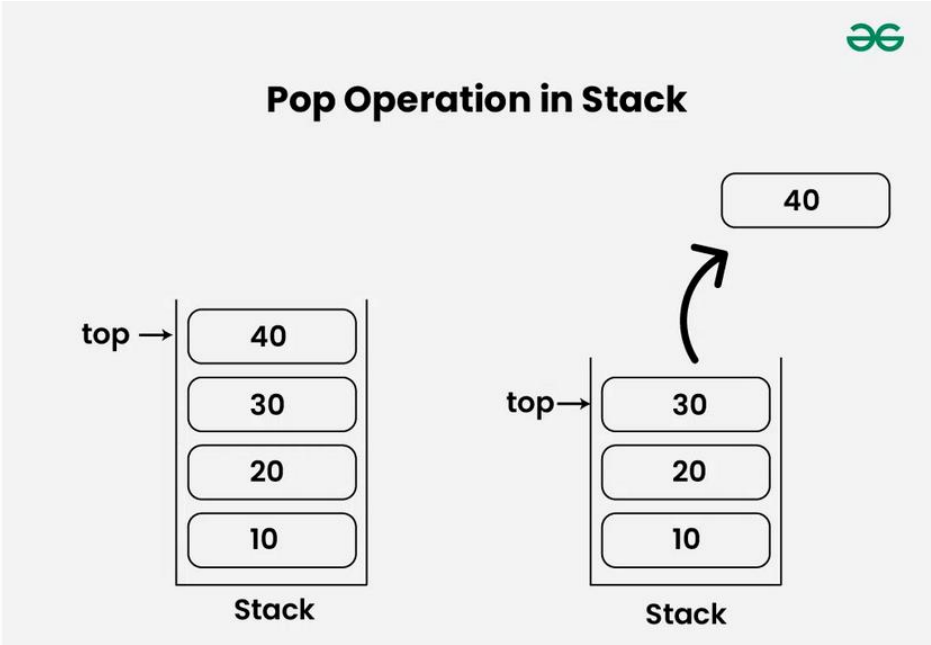
STACK

Un stack (pila) es una estructura de datos que sigue el principio LIFO (Last In, First Out), que significa “el último en entrar es el primero en salir”. Esto implica que el elemento más reciente

que se inserta en la pila será el primero en eliminarse, mientras que el primer elemento que se insertó será el último en salir. Este comportamiento se logra porque tanto la inserción como la eliminación de elementos se realizan únicamente en un extremo de la estructura, conocido como la parte superior de la pila o top.[3]



En cuanto al acceso a los elementos, en una pila solo es posible acceder directamente al elemento que se encuentra en la parte superior. Para obtener ese valor se utiliza el método top(), que permite consultar el elemento que está arriba sin eliminarlo de la pila.[3]



Respecto a la eliminación de elementos, en una pila también solo se puede eliminar el elemento que se encuentra en la parte superior. Esta operación se realiza utilizando el método pop(), el cual retira el elemento que está en el top en una sola operación. De esta manera se mantiene el comportamiento característico de la pila, donde siempre se trabaja con el último elemento que fue agregado.[3]

Ejemplo

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main(){
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    st.push(40);

    // Accessing the top element
    cout << st.top();
    return 0;
}
```

Output

```
40
```

FORWARD LIST

El forward_list en la biblioteca estándar de C++ (STL) es un contenedor que implementa la estructura de datos conocida como lista enlazada simple (singly linked list). En este tipo de estructura, los elementos no se almacenan en posiciones de memoria contiguas. En su lugar, cada elemento guarda un enlace o referencia al siguiente elemento de la lista. Gracias a esta forma de organización, las operaciones de inserción y eliminación pueden realizarse de manera rápida una vez que se conoce la posición del elemento dentro de la lista.[4]

En C++, el forward_list se define mediante la plantilla de clase std::forward_list, que se encuentra en el archivo de encabezado <forward_list>. Para utilizarlo, primero se debe incluir ese encabezado y luego declarar una lista indicando el tipo de datos que almacenará.[4]

La sintaxis básica es la siguiente:

```
forward_list<T> fl;
```

Aquí, T representa el tipo de dato que almacenarán los elementos de la lista, mientras que fl es el nombre que se le asigna a la estructura. De esta manera, se puede crear una lista enlazada simple que almacene elementos del tipo especificado.

Ejemplo

```cpp
#include <bits/stdc++.h>
using namespace std;

void printFL(forward_list<int>& fl) {
    for (auto i : fl)
        cout << i << " ";
    cout << '\n';
}

int main() {

    forward_list<int> fl1;
    forward_list<int> fl2(3, 4);
    forward_list<int> fl3 = {1, 5, 3, 4};

    printFL(fl2);
    printFL(fl3);
    return 0;
}
```

Output

```
4 4 4
1 5 3 4
```